

Neural Network Basics in Sports Statistics

Vinnie Carbone

Neural Networks in Sports Analytics

What is a Neural Network?

- A neural network is a statistically trained model that draws inspiration from the human brain
- It learns from data in order to make predictions
- It is not explicitly programmed, the key here is that it **learns**

Why Use Neural Networks?

- Able to see **complex relationships** in data
 - Works best with nonlinear patterns
- Used commonly in many fields including:
 - Image recognition
 - Speech recognition
 - Sports analytics

Today's Goal

- Build a simple neural network to predict NBA player performance. We will predict:
 - Points, rebounds, and assists.
- Compare predicted vs actual stats to see how well the neural network predicts

How Neural Networks Work

Basic Structure

- Neural networks are made up of layers of neurons (the human brain has billions of neurons)
 - Input layer
 - Hidden layers
 - Output layer
- Each neuron (or node):
 - Takes in numbers
 - Applies a transformation
 - Passes the result forward

Flow of Information

- Data moves **forward** through the network
- Each layer transforms the data slightly
- Final layer produces a prediction

Simple Diagram

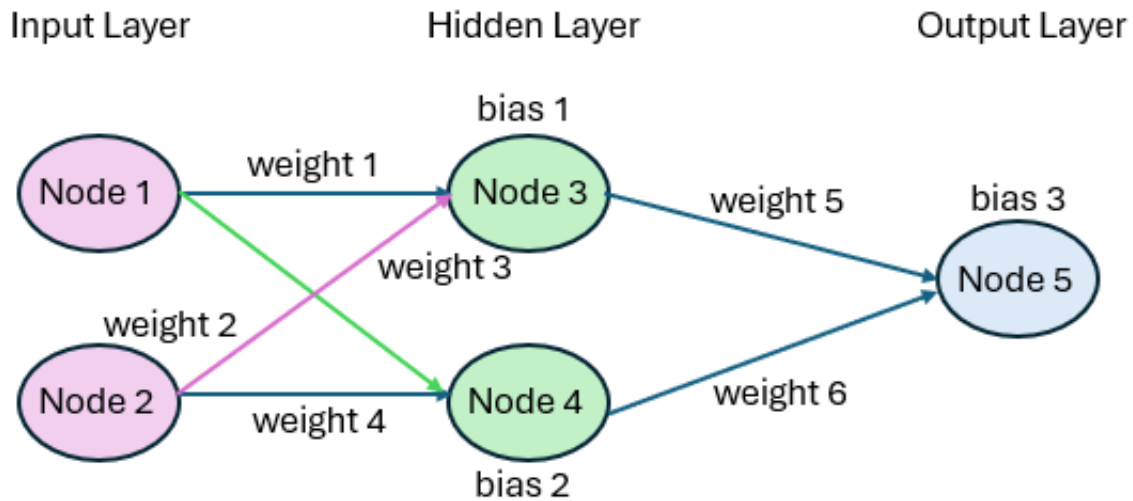


Figure 1: Neural network diagram

What Happens Inside a Neural Network?

Neurons (Nodes)

- Each circle in the network is called a **neuron/node**
- A neuron:
 - Receives inputs
 - Processes them
 - Passes an output forward

Weights and Bias

- **Weights:**
 - Control how important each input is
 - Larger weight = more influence on the prediction
- **Bias:**
 - A constant added to help shift predictions
 - Allows the model to be more flexible

Activation Function (ReLU)

- After combining inputs, neurons apply an **activation function**
- We used **ReLU (Rectified Linear Unit)**:

$$f(x) = \max(0, x)$$

- This means:
 - If input is negative $\rightarrow$ output is 0
 - If input is positive $\rightarrow$ keep it

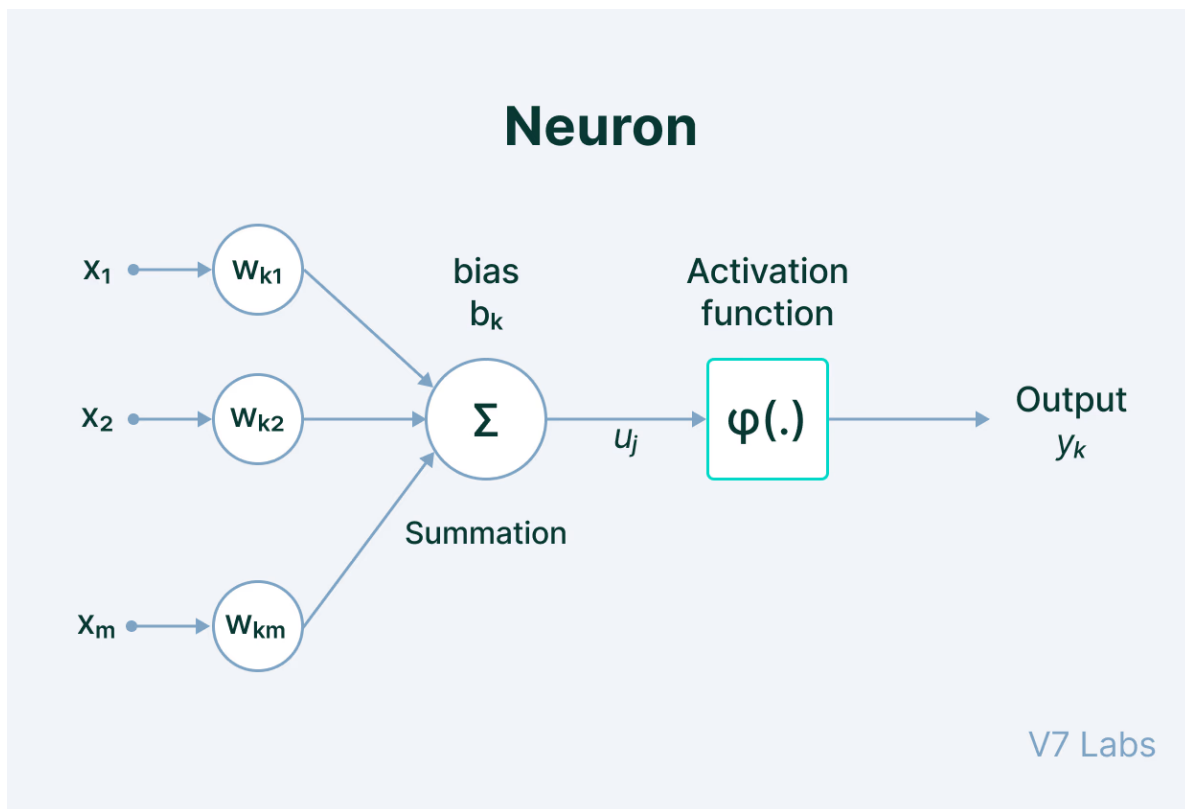


Figure 2: Neuron diagram

Why This Matters

- ReLU helps the model learn **nonlinear patterns**
- This is important because:
 - Player performance is not perfectly linear

Other Activation Functions

Sigmoid

- Outputs values between **0 and 1**
- Often used for probabilities

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Limitation:
 - Can slow down learning in deeper networks

Tanh (Hyperbolic Tangent)

- Outputs values between **-1 and 1**
- Centered at 0 (helps learning compared to sigmoid)

$$f(x) = \tanh(x)$$

- Still can have issues with slow learning in deep networks

Key Takeaway

- Different activation functions change how the model learns patterns
- ReLU is most common, but others are useful in specific situations

Example of a Neural Network in Sports Statistics

Data

- Using Basketball-Reference.com, I imported these eight stats from players in the NBA since the 2019 season
- Data cleaning: 0 and NA are not the same thing in this context

Click to expand code

```
lebron_df = all_df[all_df["player"] == "LeBron James"] \
    .sort_values("season")[
        ["player", "season", "age", "mpg", "games", "fga", "efg", "pts", "trb", "ast",]
    ]
lebron_df
```

	player	season	age	mpg	games	fga	efg	pts	trb	ast
2217	LeBron James	2019	34	35.2	55	19.9	0.560	27.4	8.5	8.3
13	LeBron James	2020	35	34.6	67	19.4	0.550	25.3	7.8	10.2
545	LeBron James	2021	36	33.4	45	18.3	0.576	25.0	7.7	7.8
1609	LeBron James	2022	37	37.2	56	21.8	0.590	30.3	8.2	6.2
1077	LeBron James	2023	38	35.5	55	22.2	0.549	28.9	8.3	6.8
3297	LeBron James	2024	39	35.3	71	17.9	0.599	25.7	7.3	8.3

Model Structure

- Input features:
 - Previous season stats: age, minutes per game (mpg), field goal attempts (fga), effective field goal percentage (efg), points (pts), rebounds (trb), assists (ast)
- Output:
 - Predicted points, rebounds, and assists for the next season

Click to expand code

```
lebron_df_2 = all_df[all_df["player"] == "LeBron James"] \
    .sort_values("season")[
        ["player", "season", "prev_age", "prev_mpg", "prev_games", "prev_fga", "prev_efg", "prev_pts", "prev_ast",]
    ]
lebron_df_2
```

	player	season	prev_age	prev_mpg	prev_games	prev_fga	prev_efg	prev_pts	prev_ast
2217	LeBron James	2019	33.0	36.9	82.0	19.3	0.590	27.5	8.6
13	LeBron James	2020	34.0	35.2	55.0	19.9	0.560	27.4	8.5
545	LeBron James	2021	35.0	34.6	67.0	19.4	0.550	25.3	7.8
1609	LeBron James	2022	36.0	33.4	45.0	18.3	0.576	25.0	7.7
1077	LeBron James	2023	37.0	37.2	56.0	21.8	0.590	30.3	8.2
3297	LeBron James	2024	38.0	35.5	55.0	22.2	0.549	28.9	8.3

Inputs & Targets

- We started with stats from 2018 - 2024. We dropped the first year, because this year has no 'previous season'.

Architecture

- Input layer $\rightarrow$ 8 features
- Hidden Layer 1 $\rightarrow$ 64 neurons (ReLU)
- Hidden Layer 2 $\rightarrow$ 32 neurons (ReLU)
- Output Layer $\rightarrow$ 3 neurons (PTS, TRB, AST)

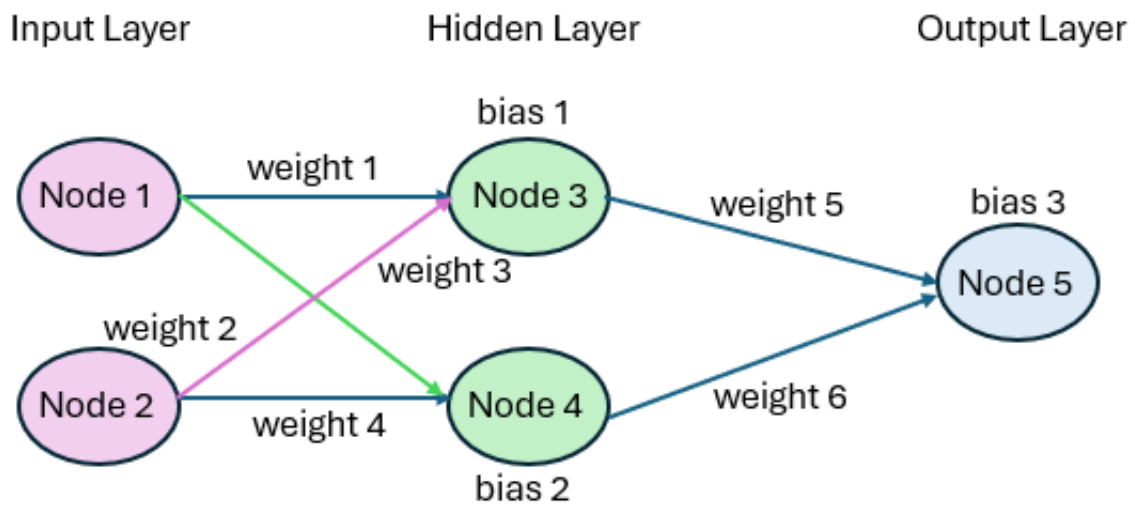


Figure 3: Neural network diagram

Why 64 and 32?

- More neurons = more ability to learn patterns
- First layer (64):
 - Captures complex relationships in player stats
- Second layer (32):
 - Refines and simplifies those patterns

Training the Model

- The model learns by adjusting:
 - Weights
 - Biases

Click to expand code

```
# Scaling
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

- Random training/testing split, rows are shuffled before splitting so the model sees different players and different seasons
- **random_state=42** sets the random seed just so the data splits the same time every run.

```
# Train/test split

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size = 0.2,
    random_state = 42
)
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(2120, 8)
(531, 8)
(2120, 3)
(531, 3)
```

Epochs

- Used TensorFlow library which has tools to create, train, and optimize neural networks
- We trained for 50 **epochs**
 - Each epoch = one full pass through the training data
 - Each epoch looks through the data 32 rows at a time (batch_size)
- **Train Loss** = error on the data the model is learning from (learning)
- **Validation Loss** = error on the data the model has NOT trained on (generalization)

Back Propagation

- **What to change**
- Determines how much each weight contributes to the prediction error
- Works **backwards through the network** (output → input)
- Computes gradients (how each weight should change)
- Happens automatically during `model.fit()`
- Provides the information needed to update the weights

Stochastic Gradient Descent

- **How to change it**
- Uses the gradients from backpropagation to update weights
- Adjusts weights to reduce prediction error
- Runs repeatedly during training
- Uses **small batches of data (32 rows)** instead of all data at once
- The optimizer = “adam” is an advanced form of gradient descent that improves efficiency
- Mean Squared Error (MSE/Train Loss) as the loss function to train the model, and Mean Absolute Error (MAE) as a metric to interpret how far off our predictions were

Key Idea

- The network learns patterns like:
 - “Higher minutes + higher usage → more points”
 - “Aging + lower minutes → possible decline”

Results and Model Performance

Overall Accuracy

- $MAE = [\text{pts error} + \text{trb error} + \text{ast error}] / 3$
 - The model is off by about 1.4 stats per game on average

Model MSE: 5.668

Model MAE: 1.625

Model RMSE: 2.380687829488232

Jaylen Brown

Predicted → PTS: 27.77, TRB: 9.11, AST: 5.42

Actual → PTS: 23.00, TRB: 5.50, AST: 3.60

Jayson Tatum

Predicted → PTS: 32.07, TRB: 12.05, AST: 7.17

Actual → PTS: 26.90, TRB: 8.10, AST: 4.90

Jrue Holiday

Predicted → PTS: 19.76, TRB: 6.22, AST: 7.66

Actual → PTS: 12.50, TRB: 5.40, AST: 4.80

Sam Hauser

Predicted → PTS: 6.81, TRB: 2.99, AST: 1.19

Actual → PTS: 9.00, TRB: 3.50, AST: 1.00

Kristaps Porziņģis

Predicted → PTS: 23.43, TRB: 10.17, AST: 4.47

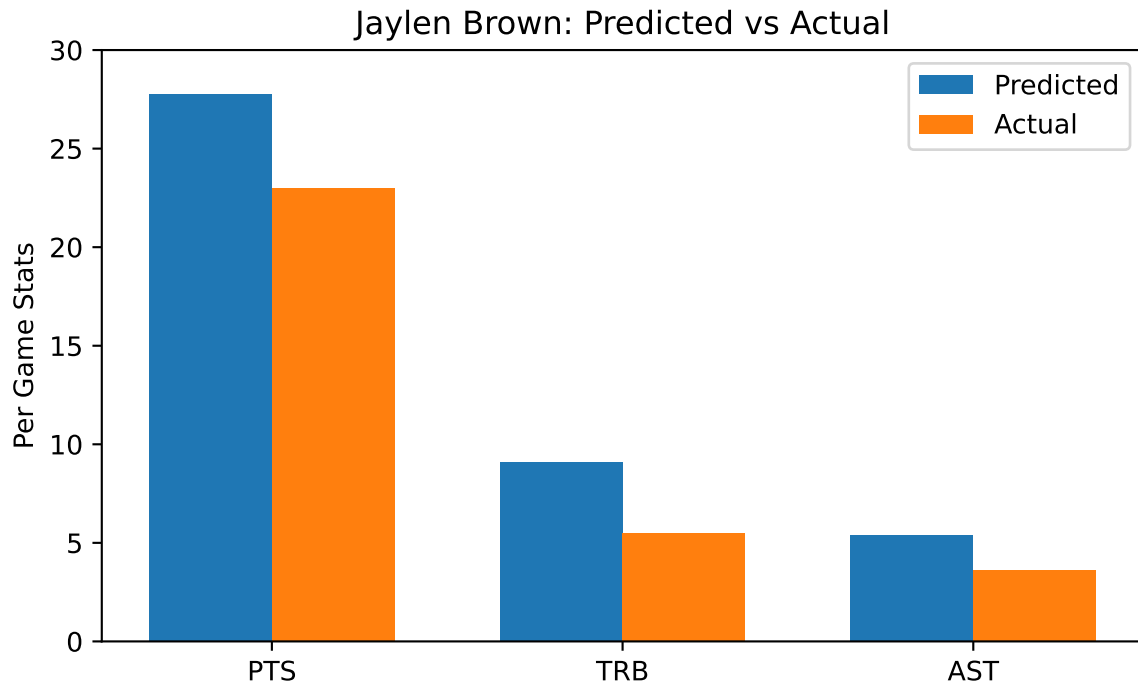
Actual → PTS: 20.10, TRB: 7.20, AST: 2.00

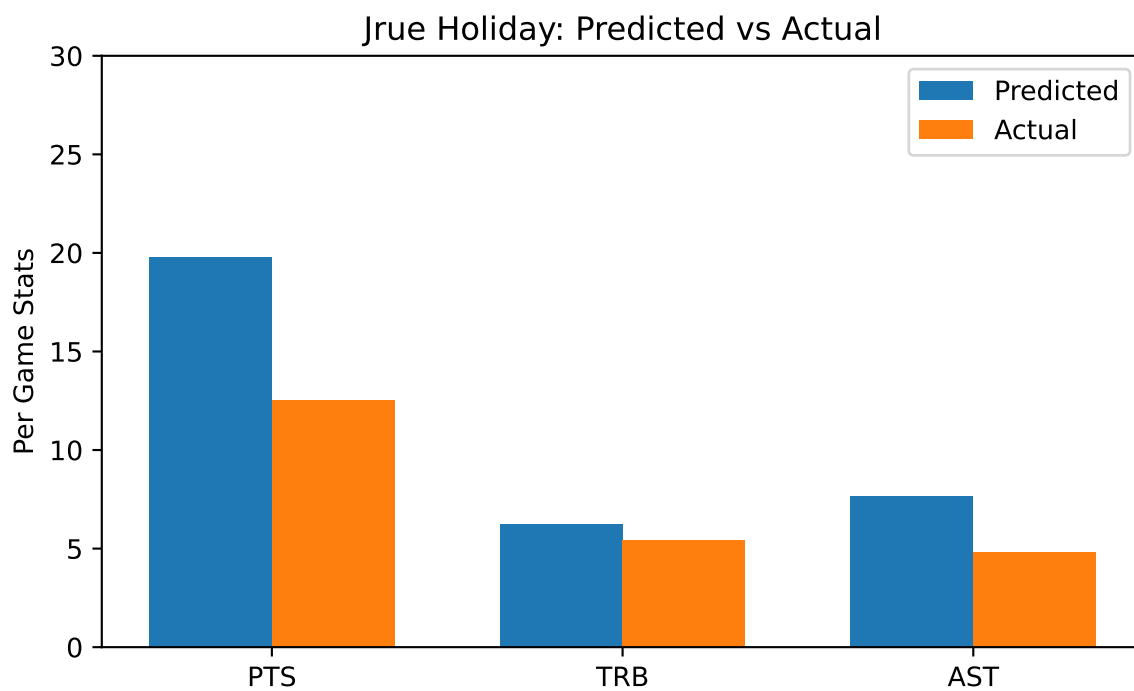
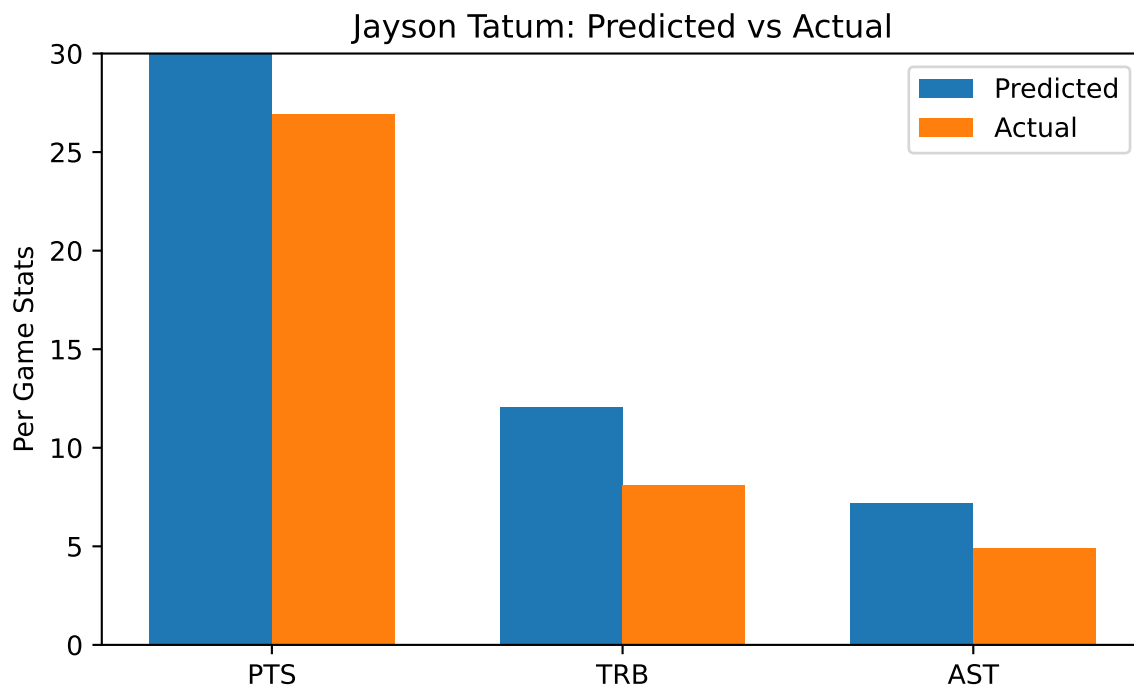
Luke Kornet

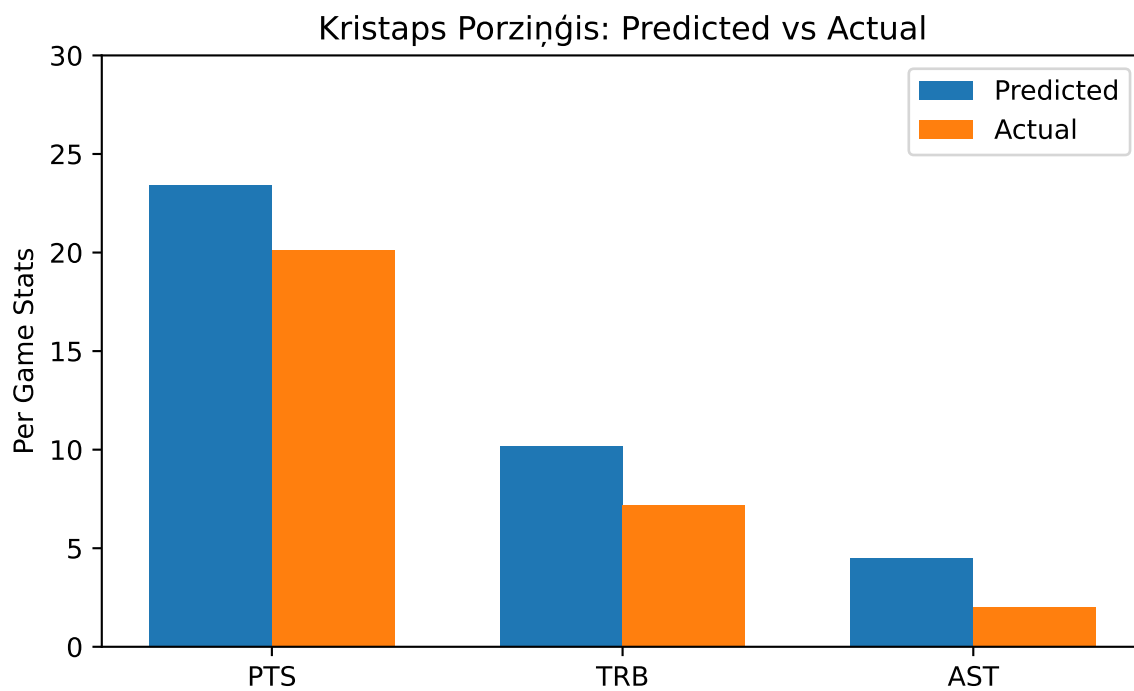
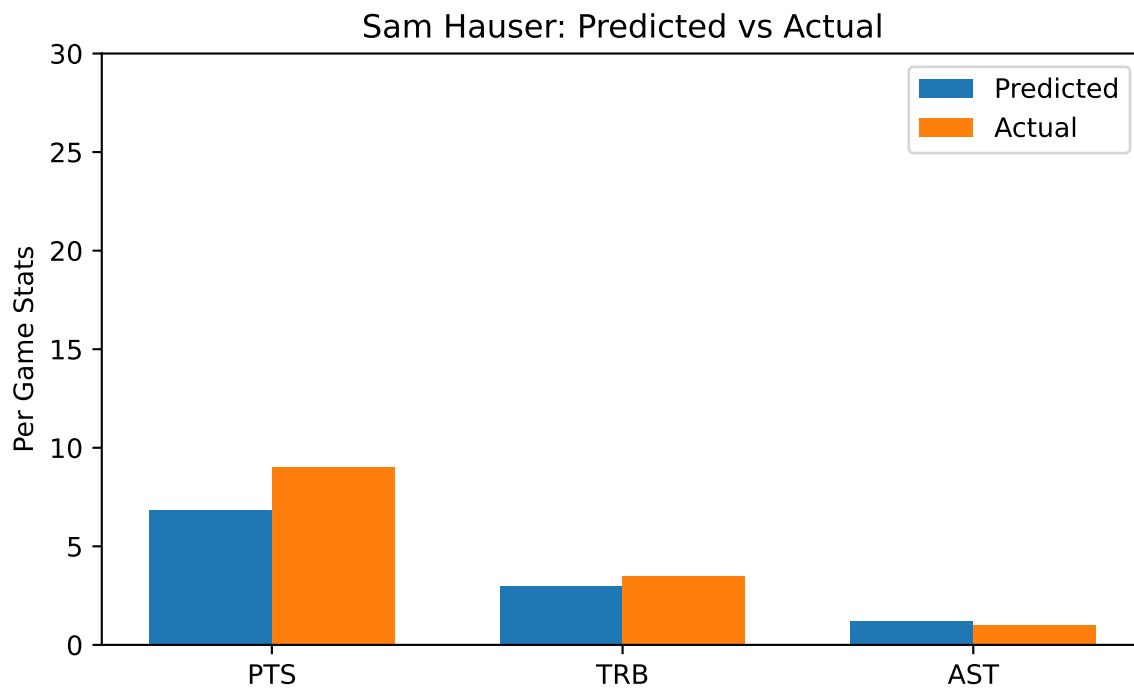
Predicted -> PTS: 3.46, TRB: 2.70, AST: 0.63
Actual -> PTS: 5.30, TRB: 4.10, AST: 1.10

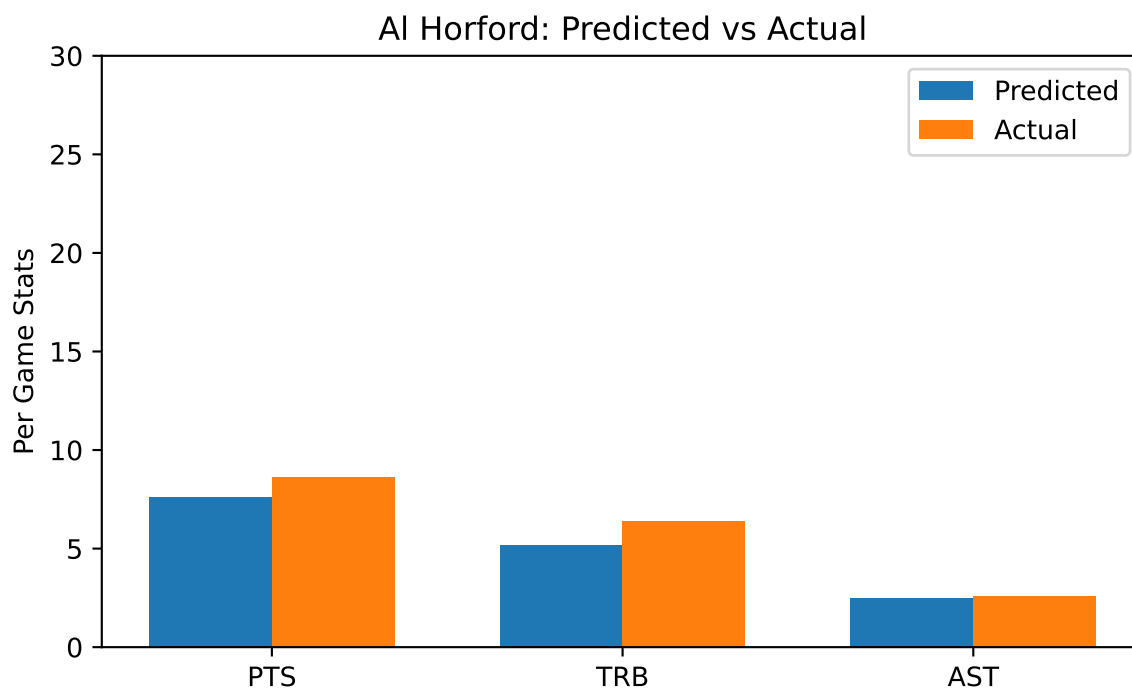
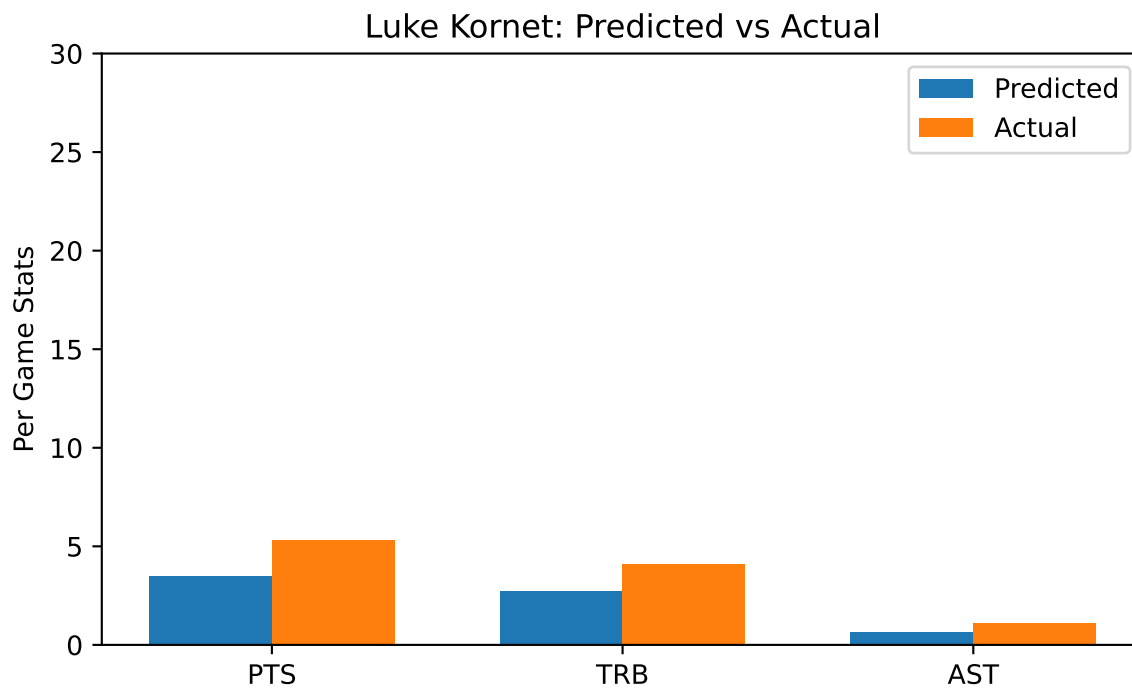
Al Horford

Predicted -> PTS: 7.61, TRB: 5.15, AST: 2.47
Actual -> PTS: 8.60, TRB: 6.40, AST: 2.60









Limitations of This Specific Model

- Only used basic statistics:
 - Age, minutes, shooting, previous stats, etc
 - Does NOT include: injuries, team changes, role changes, etc
- Dataset is relatively small
 - Fewer examples can limit how well the model learns
- Simple neural network (64 → 32 layers)
 - More complex models could improve performance
- Does not account for:
 - Team system
 - Teammates
 - Coaching changes

Limitations of Neural Networks in General

- Neural networks require **large amounts of data**
 - Performance suffers with limited or noisy data
- Often difficult to interpret
 - Difficult to understand how predictions are made
- Training can be:
 - Time-consuming
 - Resource-intensive

Further Reading

- [Other applications of neural networks](#)
- <https://course.elementsofai.com/en-ie/5/1>
- [Creating your own neural network in python](#)